Transformers and Natural Language Processing

CSCI-P 556

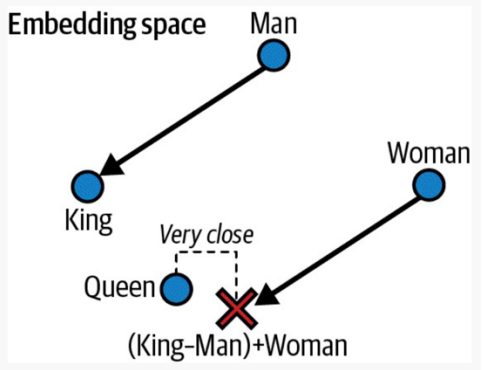
ZORAN TIGANJ

Today: Attention to transformers

- Slides based on:
  - **Transformers**
    - Ashish Vaswani, et al. "Attention is all you need" NeurIPS 2017 (almost 20000 citations in 4 years), great success in machine translation, now being applied to other domains as well, such as time series prediction.
  - Other useful resources:
    - https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html
    - https://towardsdatascience.com/attention-in-neural-networks-e66920838742
    - https://towardsdatascience.com/hands-on-memory-augmented-neural-networks-implementation-part-one-a6a4a88beba3
    - http://jalammar.github.io/illustrated-transformer/
    - Try it out: https://pytorch.org/tutorials/beginner/transformer_tutorial.html

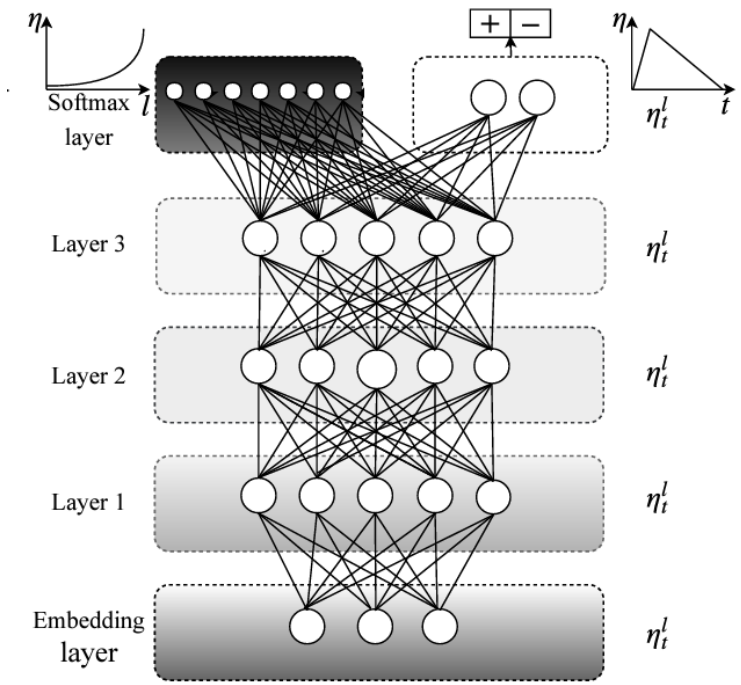
Reusing Pretrained Embeddings

- To do NLP, we want to embed words (or sentences) into meaningful vectors that will reflect relationships between the words.
  - Example: King – Man + Woman = Queen
- We can do it by starting with random embeddings and feeding them into a neural network trained to predict the next word.
- Or we can just download and use pretrained embeddings, such as Google's *Word2vec* embeddings, Stanford's *GloVe* embeddings, or Facebook's *FastText* embeddings.



Reusing Pretrained Language Models

- In the past couple of years, using pretrained language models (instead of just taking pretrained embeddings) became a norm for NLP.
  - In 2018 the Universal Language Model Fine-Tuning (ULMFiT) paper by Jeremy Howard and Sebastian Ruder demonstrated the effectiveness of unsupervised pretraining for NLP tasks: the authors trained an LSTM language model on a huge text corpus using self-supervised learning (i.e., generating the labels automatically from the data), then they fine-tuned it on various tasks.
  - Their model outperformed the state of the art on six text classification tasks by a large margin (reducing the error rate by 18–24% in most cases).
  - Moreover, the authors showed a pretrained model fine-tuned on just 100 labeled examples could achieve the same performance as one trained from scratch on 10,000 examples.
  - This is very similar to what we saw in computer vision with pretrained models.



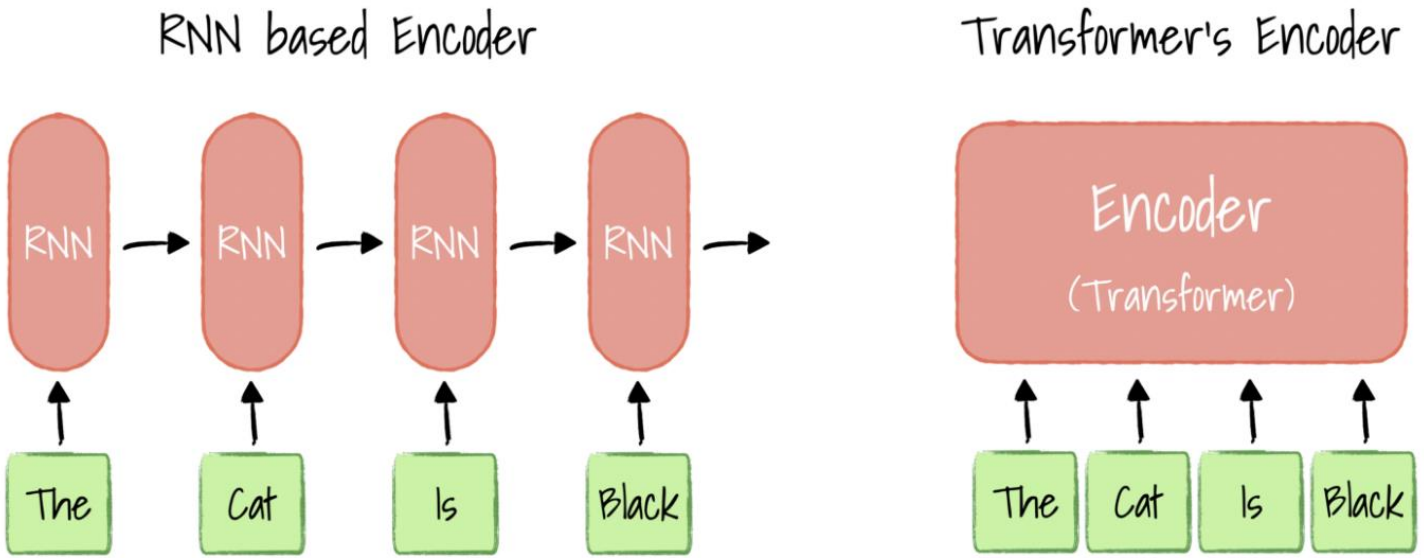
https://arxiv.org/abs/1801.06146

Reusing Pretrained Language Models

```
import os
import tensorflow_hub as hub

os.environ["TFHUB_CACHE_DIR"] = "my_tfhub_cache"
model = tf.keras.Sequential([
    hub.KerasLayer("https://tfhub.dev/google/universal-sentence-encoder/4",
                   trainable=True, dtype=tf.string, input_shape=[]),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(loss="binary_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit(train_set, validation_data=valid_set, epochs=10)
```

Big picture about transformers



RNN unroll each word individually. Transformer process input in parallel.

RNNs do a good job in natural language processing, but they have difficulty learning long-range dependencies.

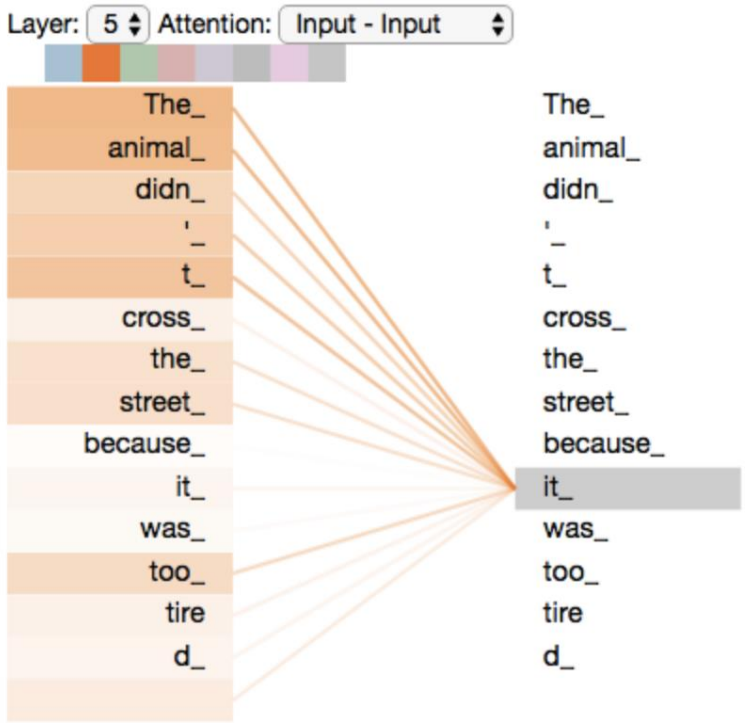
https://towardsdatascience.com/illustrated-guide-to-transformer-cf6969ffa067

Attention in Natural Language Processing

- Consider a sentence:

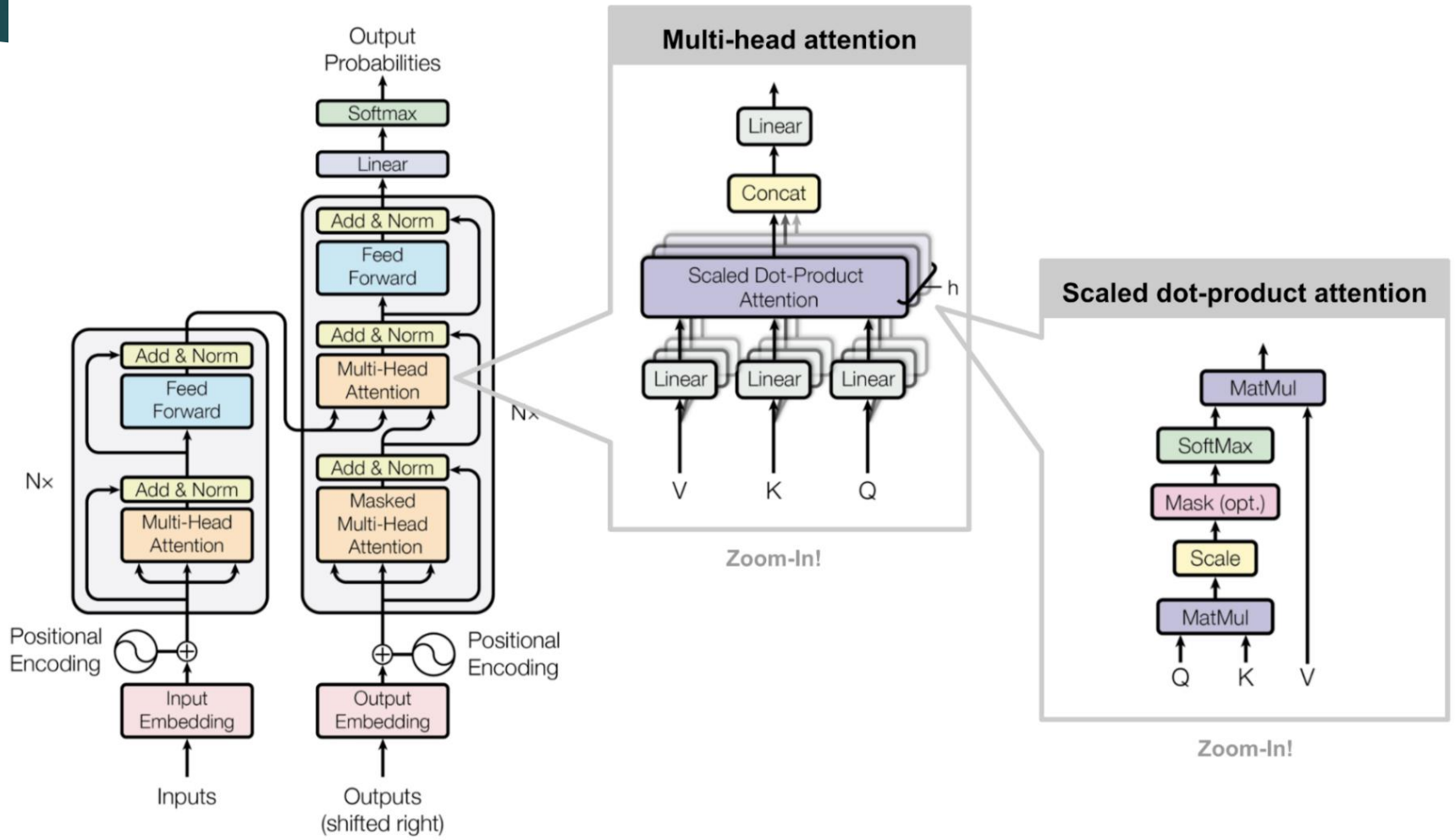
"The animal didn't cross the street because it was too tired"

  - How does a language model know what the word "it" refers to?
  - The idea of attention is to map (through weights) inputs to the relevant parts of context.
  - We will illustrate the concept of attention through **Transformers.**



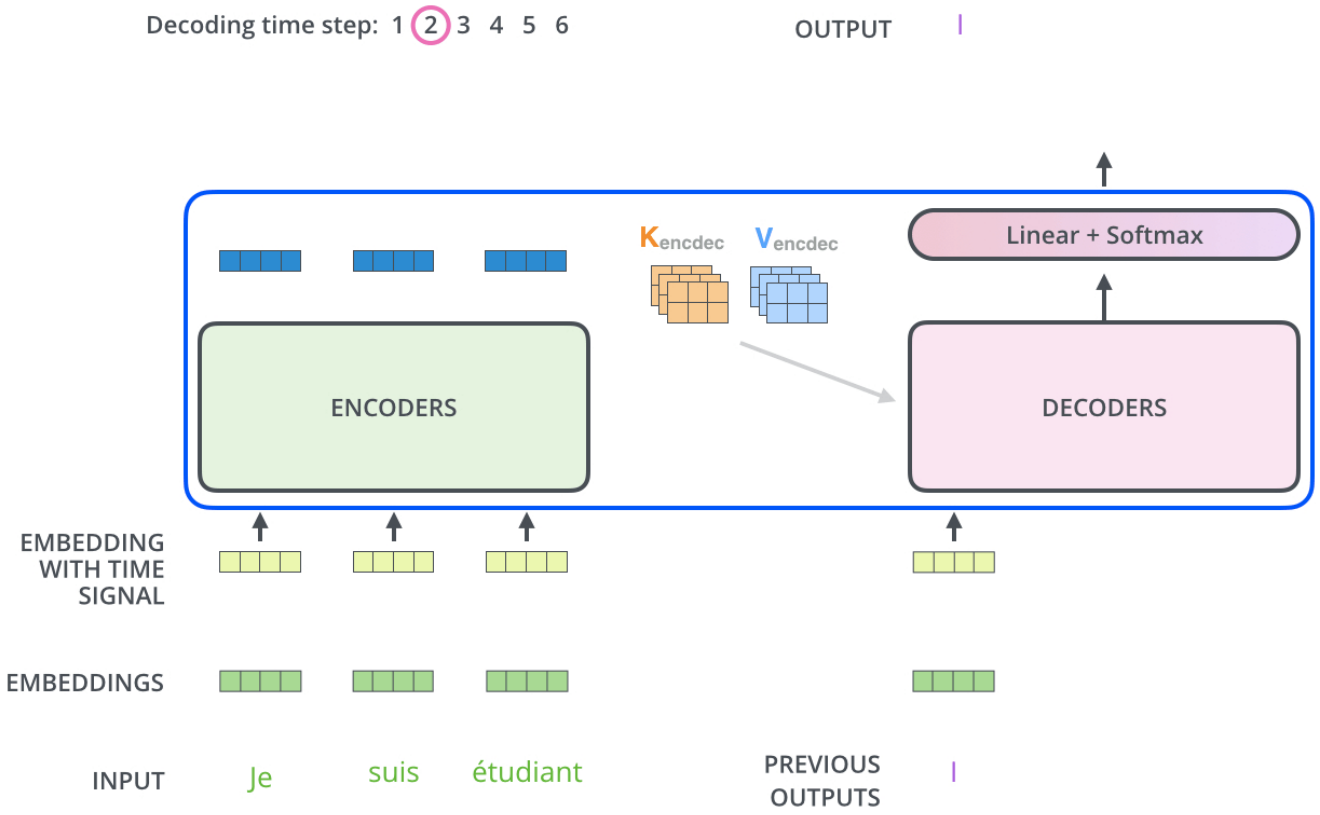
https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb#scrollTo=OJKU36QAfqOC

Transformers: Attention Is All You Need



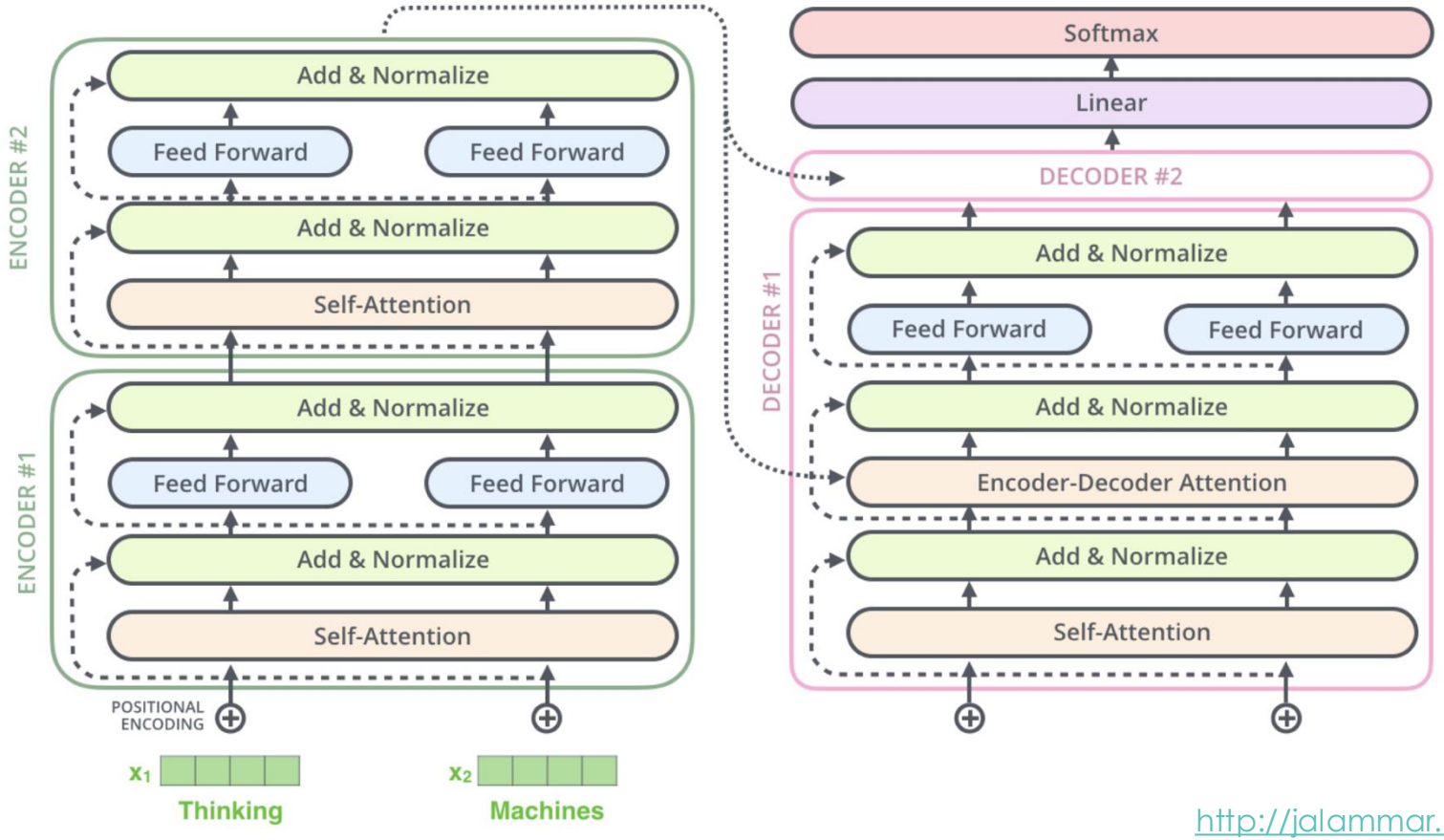
Ashish Vaswani, et al. "Attention is all you need." NeurIPS 2017.

Transformer at work

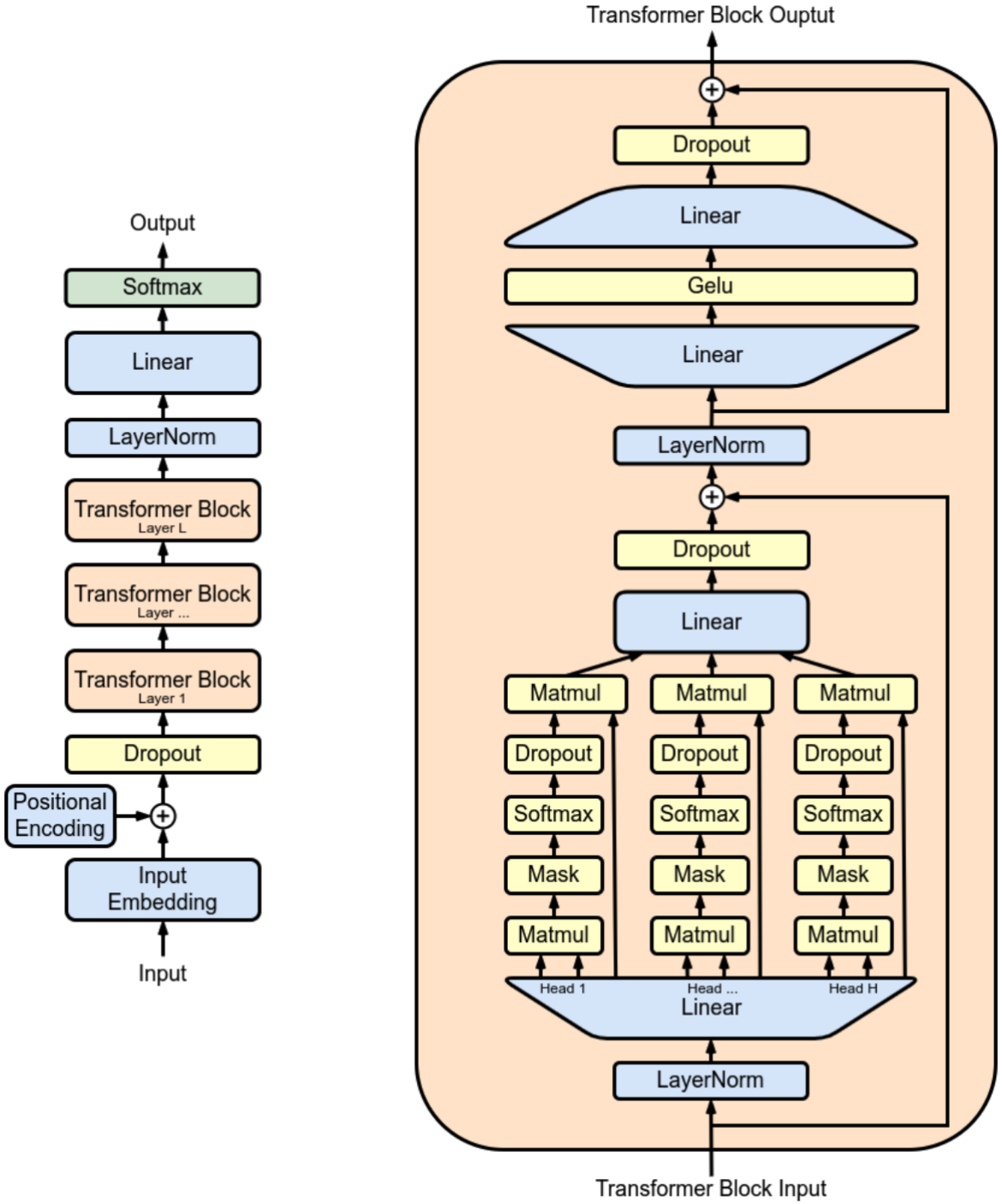


http://jalammar.github.io/illustrated-transformer/

Transformers: Attention Is All You Need



http://jalammar.github.io/illustrated-transformer/



Wikipedia (GPT)

Transformers: Attention Is All You Need

- In transformers, an attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors.
- The output is computed as a weighted sum of the values (V), where the weight assigned to each value is computed by a compatibility function of the query (Q) with the corresponding key (K):

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

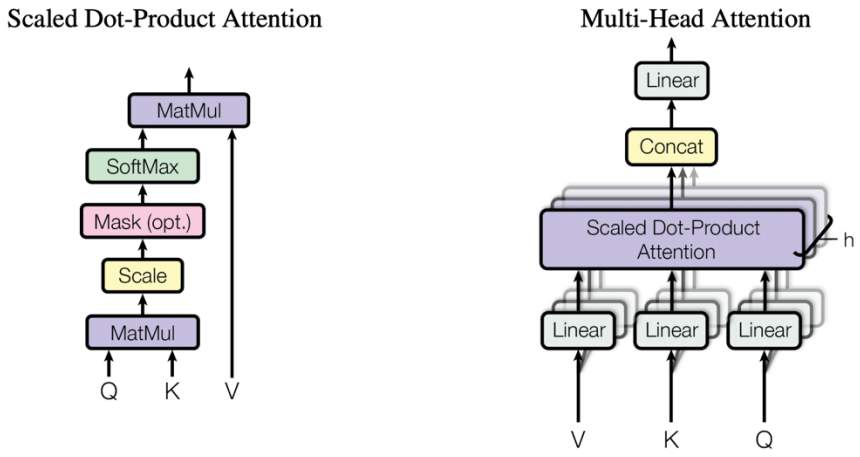
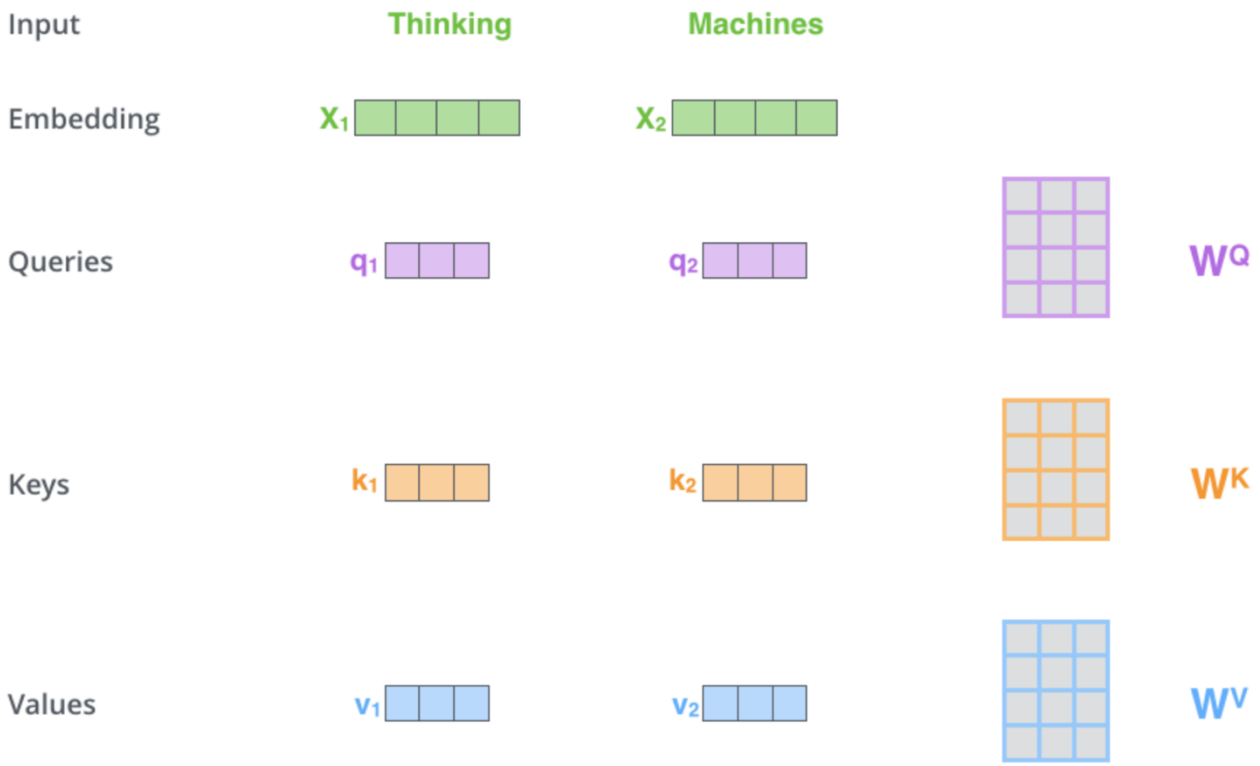


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Ashish Vaswani, et al. "Attention is all you need." NeurIPS 2017.

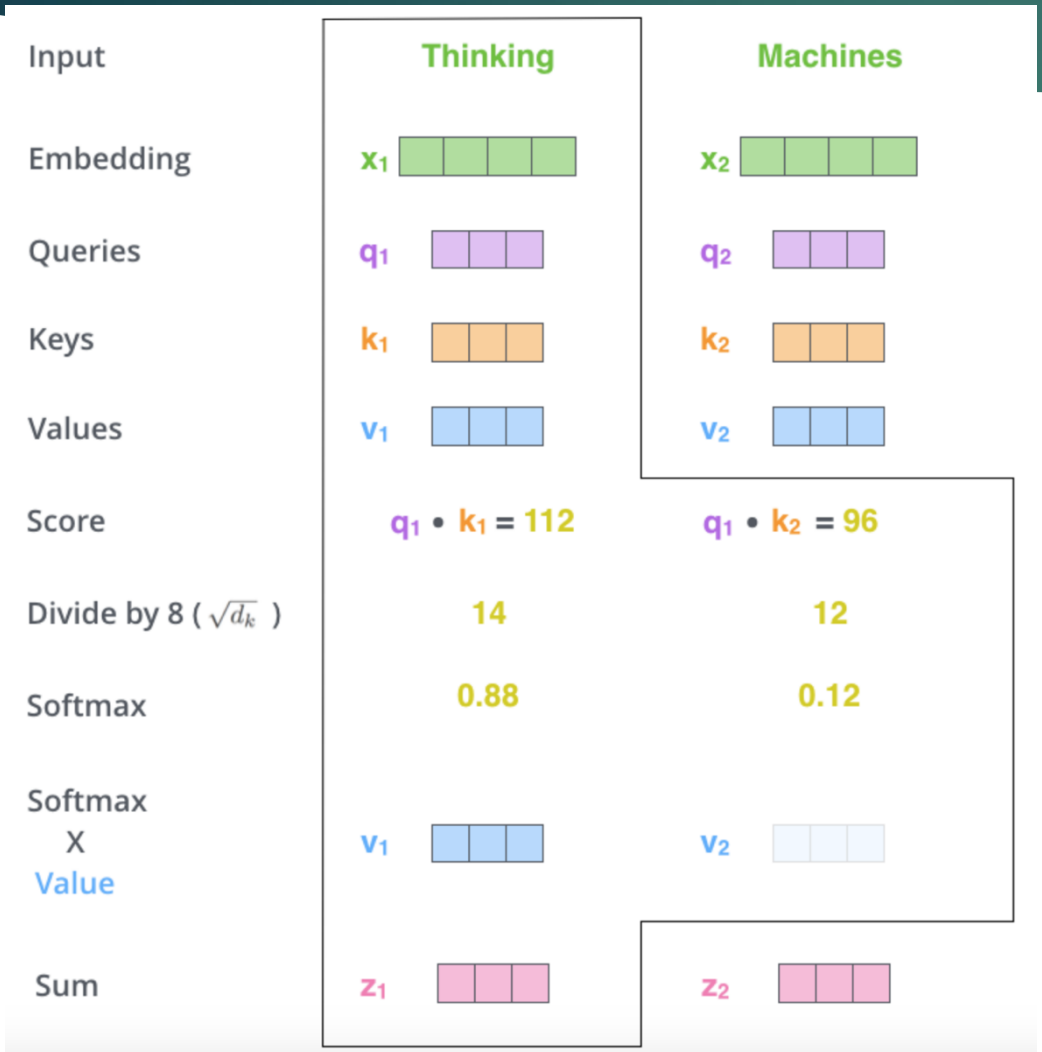
Attention in transformers



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

http://jalammar.github.io/illustrated-transformer/

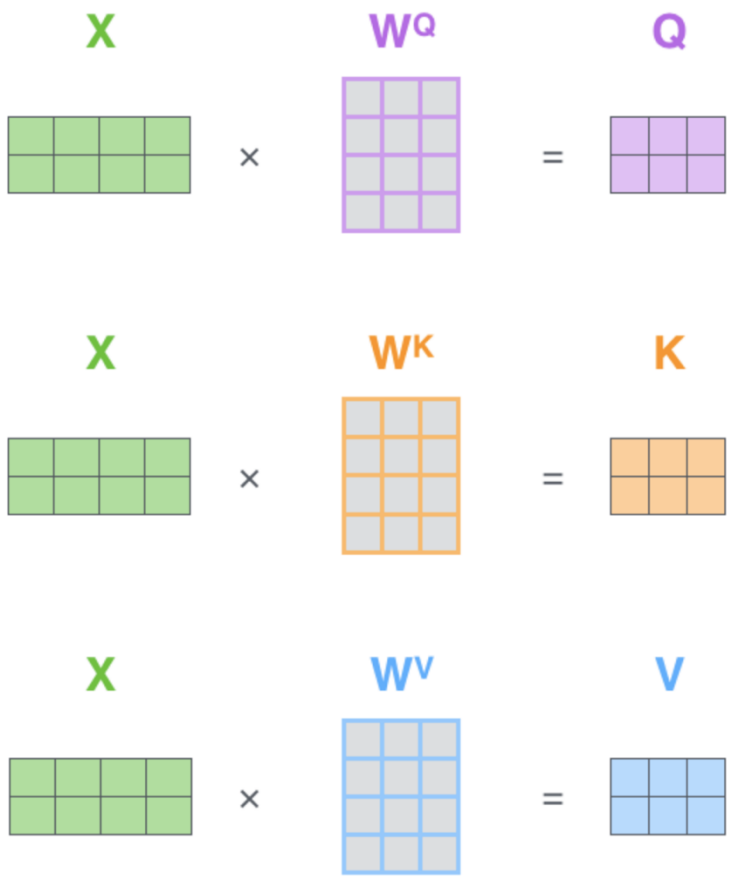
Attention in transformers

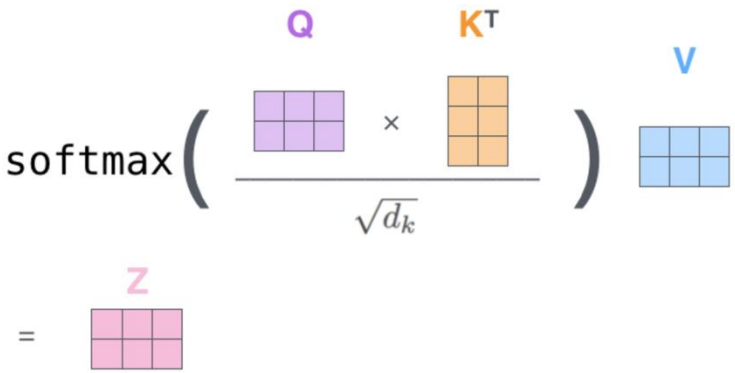


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

http://jalammar.github.io/illustrated-transformer/

Attention in transformers

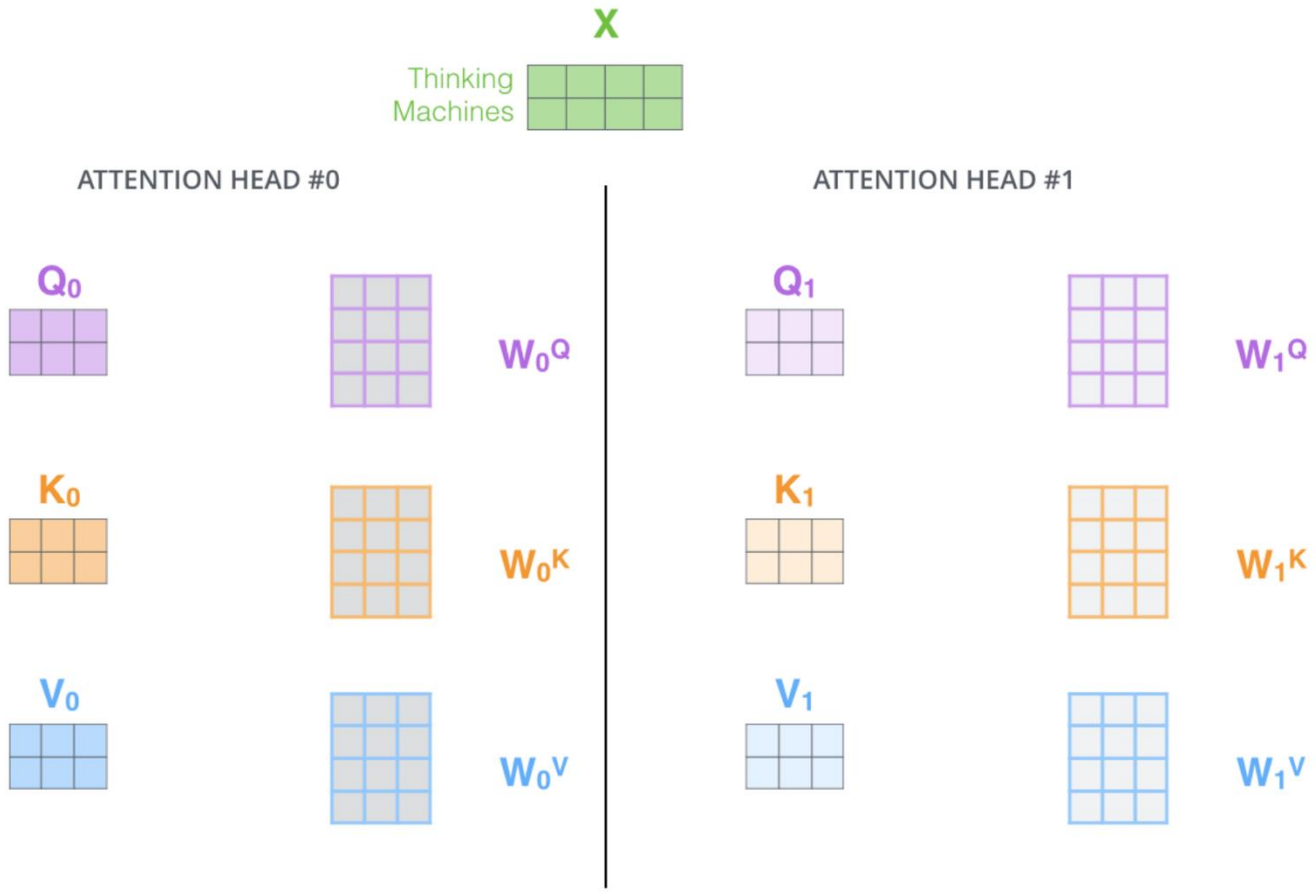




$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

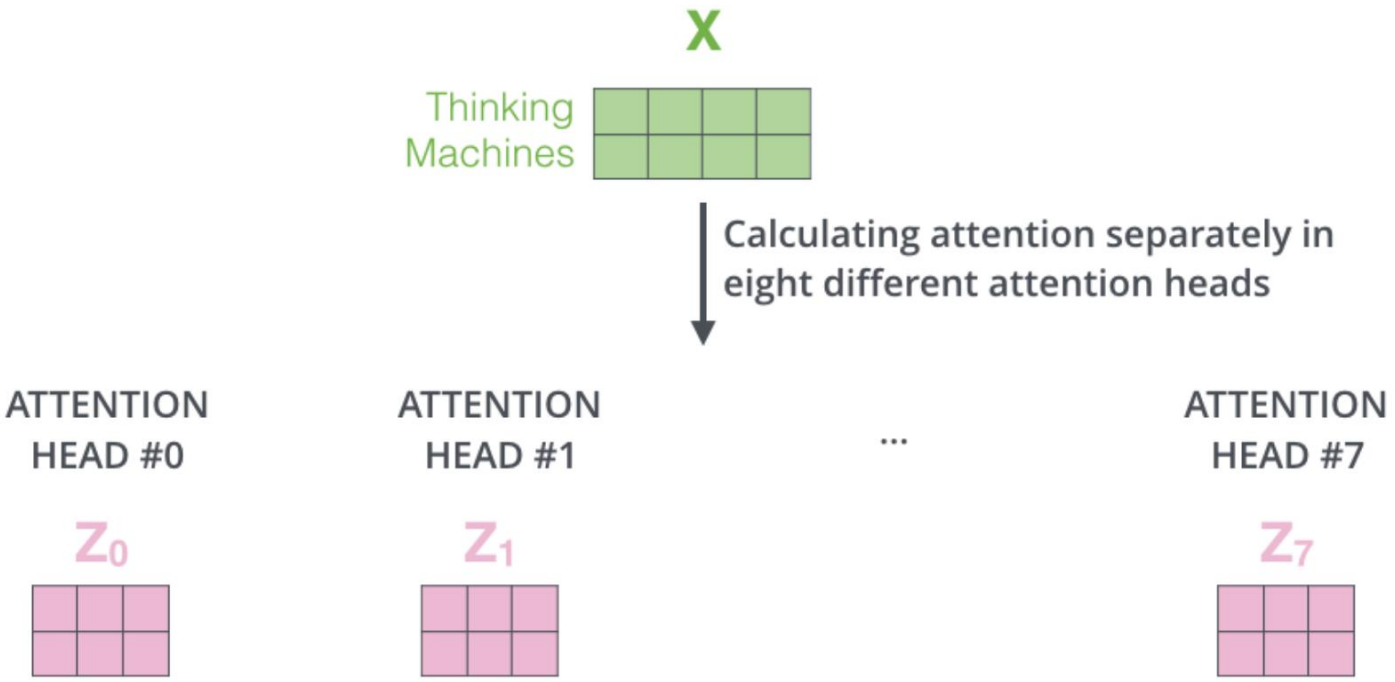
http://jalammar.github.io/illustrated-transformer/

Attention in transformers: Multiple attention heads



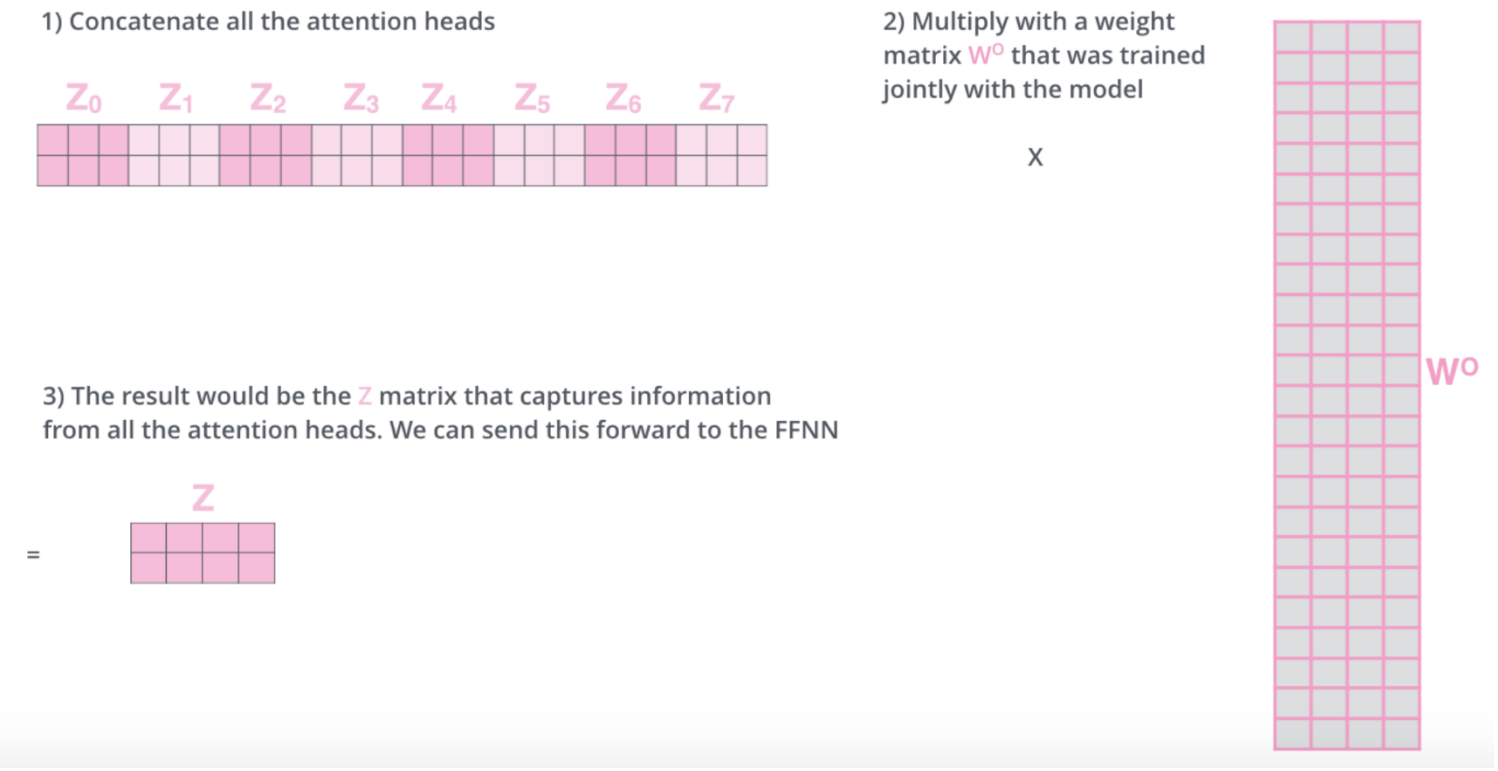
http://jalammar.github.io/illustrated-transformer/

Attention in transformers: Multiple attention heads



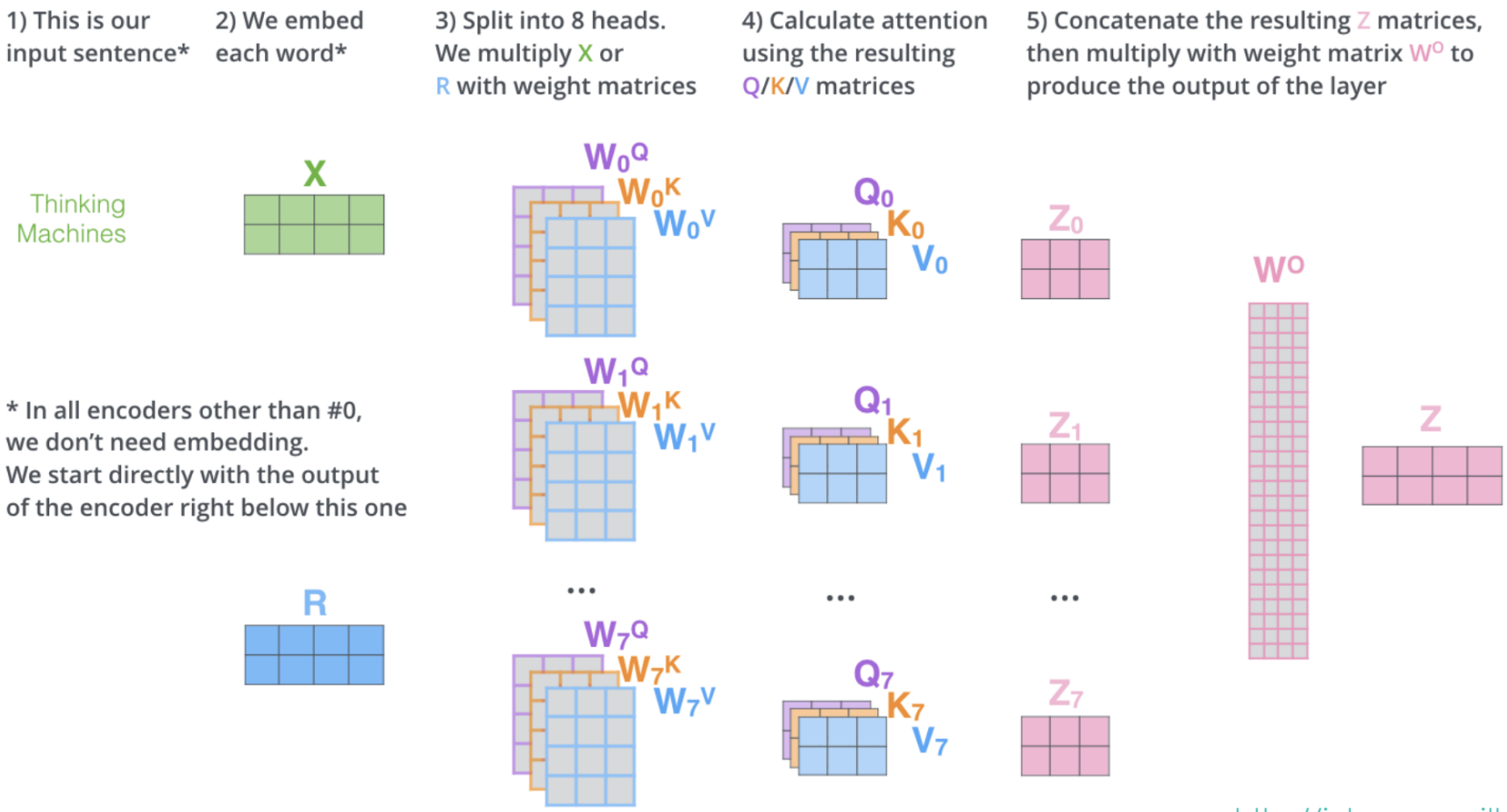
http://jalammar.github.io/illustrated-transformer/

Attention in transformers: Multiple attention heads



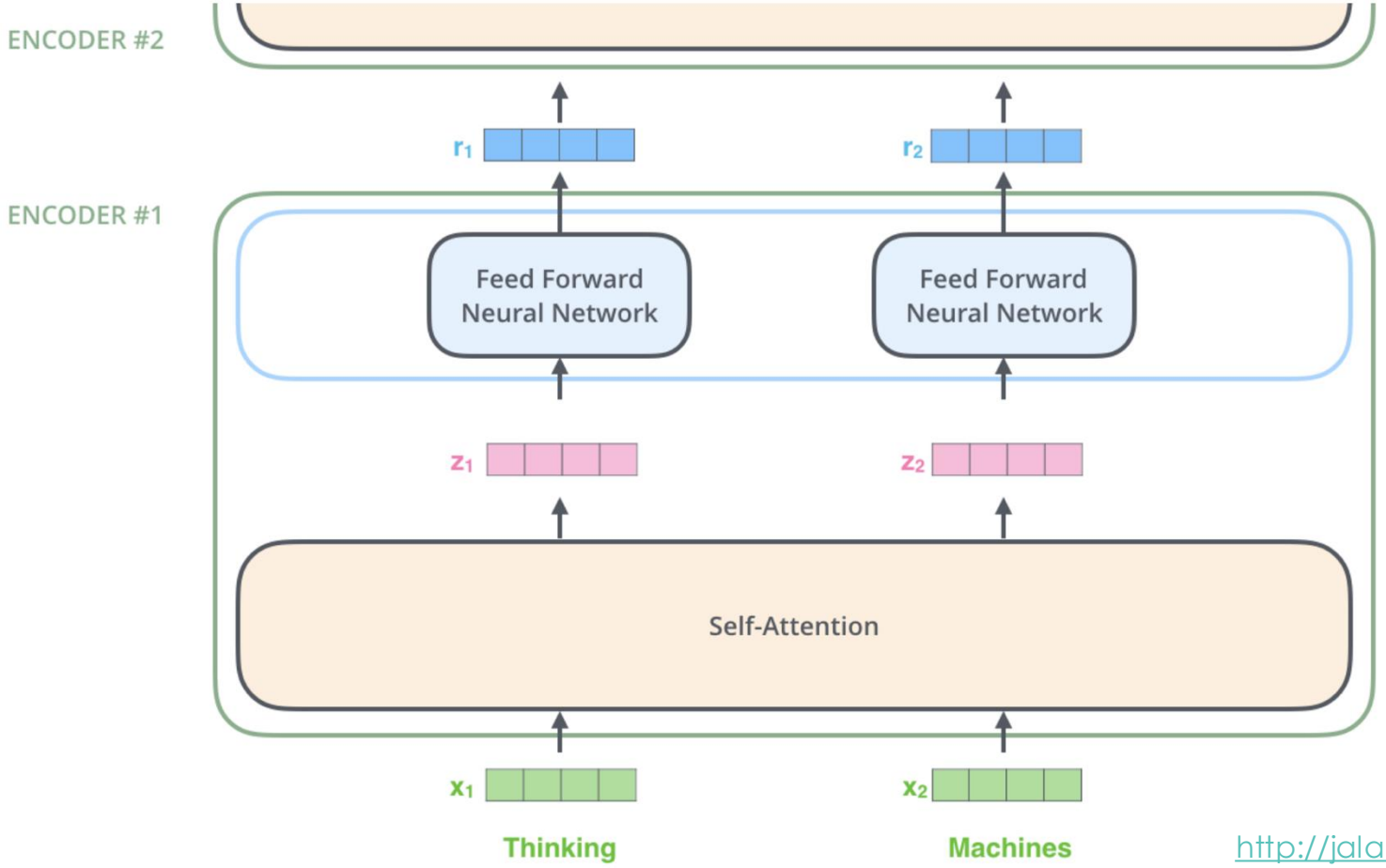
http://jalammar.github.io/illustrated-transformer/

Transformer: putting it all together



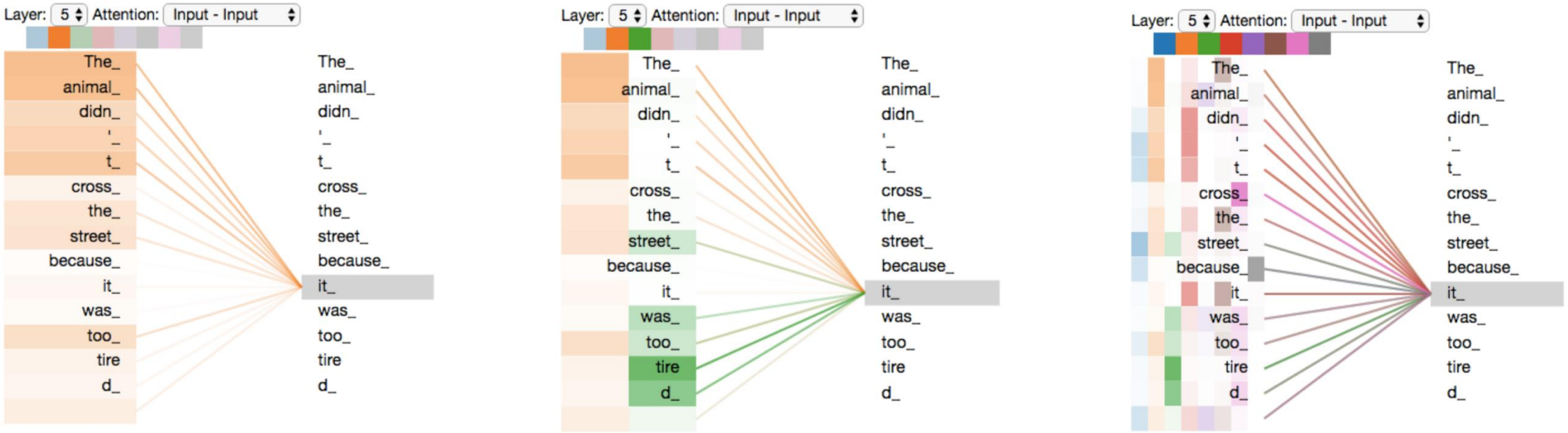
http://jalammar.github.io/illustrated-transformer/

Transformer: putting it all together



http://jalammar.github.io/illustrated-transformer/

Illustration of multihead attention in transformers



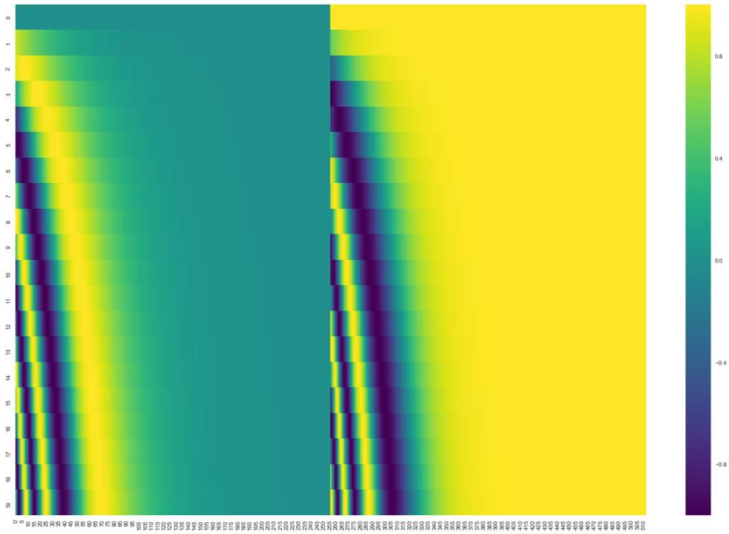
http://jalammar.github.io/illustrated-transformer/

Positional encoding in transformers

- Positional encoding:
  - "Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence."
  - The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed.
  - "In this work, we use sine and cosine functions of different frequencies:

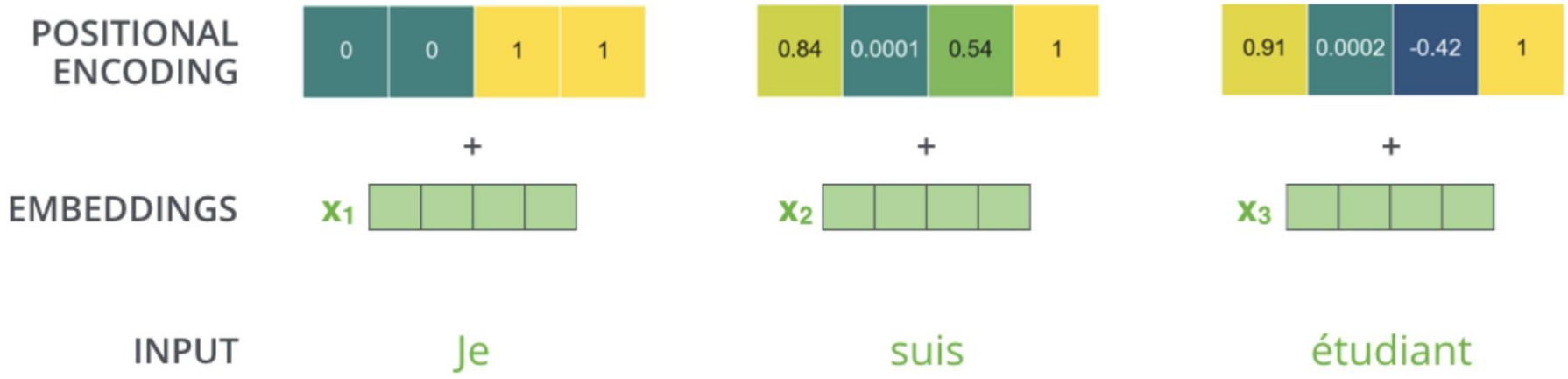
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

  - where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to 10000 · 2π."



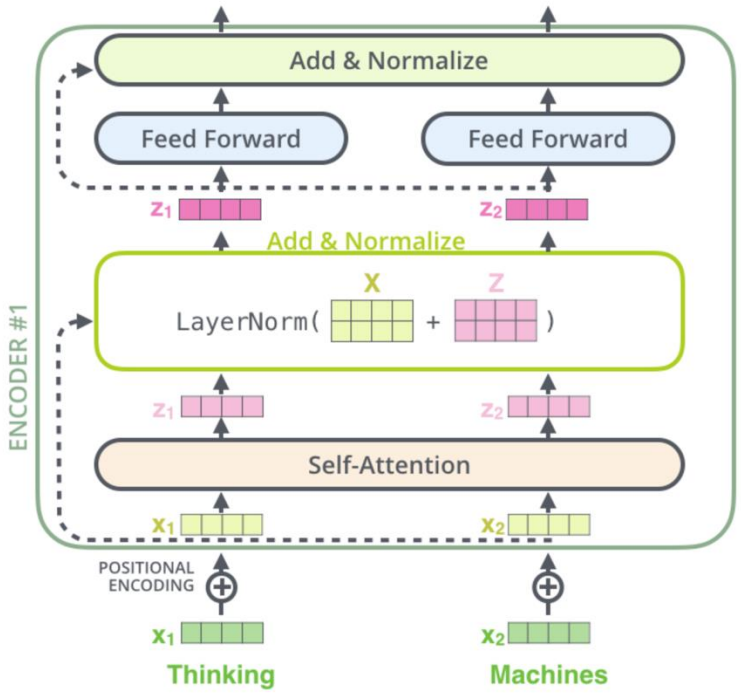
Ashish Vaswani, et al. "Attention is all you need." NeurIPS 2017.

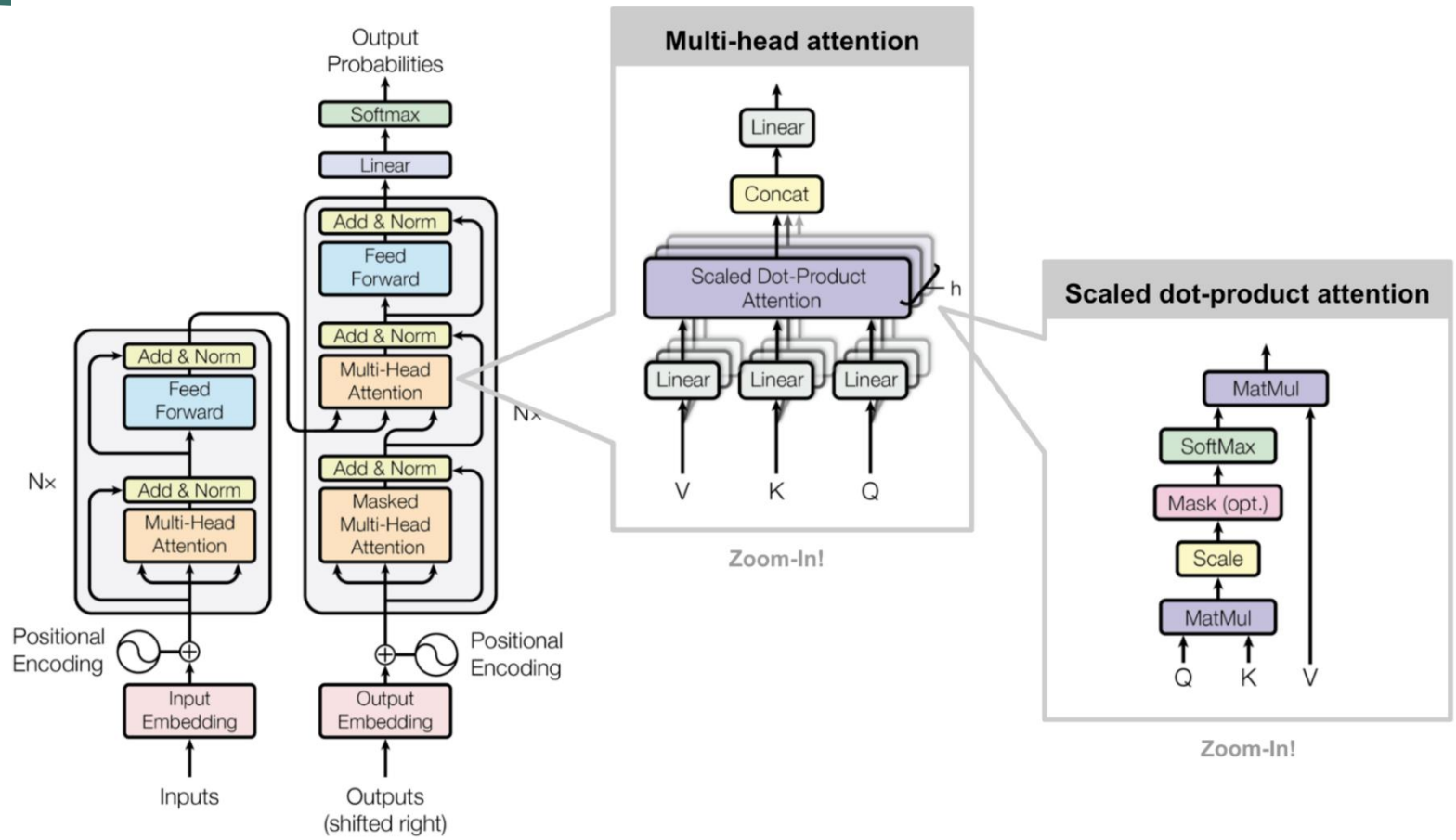
Positional encoding in transformers



http://jalammar.github.io/illustrated-transformer/

Transformer: putting it all together





http://jalammar.github.io/illustrated-transformer/

Ashish Vaswani, et al. "Attention is all you need." NeurIPS 2017.

An Avalanche of Transformer Models: GPT

- First, the GPT paper by Alec Radford and other OpenAI researchers (2018) once again demonstrated the effectiveness of unsupervised pretraining, like the ULMFiT paper before it, but this time using a transformer-like architecture.
  - They trained it on a very large dataset with the objective of just predicting the next token.
  - Then they fine-tuned it on various language tasks (e.g., question answering).

An Avalanche of Transformer Models: BERT

- Then Google's BERT paper came out (2018/2019): it also demonstrated the effectiveness of self-supervised pretraining on a large corpus, using a similar architecture to GPT but with nonmasked multi-head attention layers only, like in the original transformer's encoder.

An Avalanche of Transformer Models: GPT2

- In February 2019 Alec Radford, Jeffrey Wu, and other OpenAI researchers published the **GPT-2 paper,** which proposed a very similar architecture to GPT, but larger still (with over 1.5 billion parameters!).
- The researchers showed that the new and improved GPT model could perform **zero-shot learning (ZSL)**, meaning it could achieve good performance on many tasks without any fine-tuning.

An Avalanche of Transformer Models…

- Google's Switch Transformers (introduced in January 2021) used 1 trillion parameters, and soon much larger models came out, such as the Wu Dao 2.0 model by the Beijing Academy of Artificial Intelligence (BAII), announced in June 2021.
- Many of these models are just too big to even be used on regular hardware: they wouldn't fit in RAM, and they would be horribly slow. Also, some of them are not released publicly.

An Avalanche of Transformer Models…

- Luckily, ingenious researchers are finding new ways to downsize transformers and make them more data-efficient. For example, the DistilBERT model, introduced in October 2019 by Victor Sanh et al. from Hugging Face, is a small and fast transformer model based on BERT.
- It is available on Hugging Face's excellent model hub, along with thousands of others.
- DistilBERT was trained using distillation (hence the name): this means transferring knowledge from a teacher model to a student one, which is usually much smaller than the teacher model.
- This is typically done by using the teacher's predicted probabilities for each training instance as targets for the student.

An Avalanche of Transformer Models…

- Many more transformer architectures came out after BERT, almost on a monthly basis, often improving on the state of the art across all NLP tasks: XLNet (June 2019), RoBERTa (July 2019), StructBERT (August 2019), ALBERT (September 2019), T5 (October 2019), ELECTRA (March 2020), GPT3 (May 2020), DeBERTa (June 2020), Switch Transformers (January 2021), Wu Dao 2.0 (June 2021), Gopher (December 2021), GPT-NeoX-20B (February 2022), Chinchilla (March 2022), OPT (May 2022), **GPT4(March 2023)** and the list goes on and on.

Vision transformer

- In October 2020, a team of Google researchers released a paper that introduced a fully transformer-based vision model, called a vision transformer (ViT).
- The idea is surprisingly simple: just chop the image into little 16 × 16 squares, and treat the sequence of squares as if it were a sequence of word representations.
- The resulting sequence of vectors can then be treated just like a sequence of word embeddings: this means adding positional embeddings, and passing the result to the transformer.

Multimodal transformers

- The latest trend in transformers consists in building large multimodal models, often capable of zero-shot or few-shot learning.
- For example, OpenAI's 2021 CLIP paper proposed a large transformer model pretrained to match captions with images: this task allows it to learn excellent image representations, and the model can then be used directly for tasks such as image classification using simple text prompts such as "a photo of a cat".

Multimodal transformers

- Soon after, OpenAI announced DALL·E, capable of generating amazing images based on text prompts. The DALL·E 2, which generates even higher quality images using a diffusion model (we'll talk more about this later)

Multimodal transformers

- In April 2022, DeepMind released the Flamingo paper, which introduced a family of models pretrained on a wide variety of tasks across multiple modalities, including text, images, and videos. A single model can be used across very different tasks, such as question answering, image captioning, and more.
- Soon after, in May 2022, DeepMind introduced GATO, a multimodal model that can be used as a policy for a reinforcement learning agent.
  - The same transformer can chat with you, caption images, play Atari games, control (simulated) robotic arms, and more, all with "only" 1.2 billion parameters.
- In March 2023, Microsoft Introduced Visual ChatGPT which can edit images using text prompts.

Next time

- Autoencoders GANs Diffusion Models